

数学与计算机

计算机病毒：对角化与不动点

Willam F. Dowling

摘要 如果定义病毒是那种可以本身“复制”的程序，病毒的构造可以被看作是应用递归函数理论的习题。在这儿，我给出病毒的一个构造，包括递归函数理论的递归定理（或者是不动点定理）的证明。目的并非为了给出一个不实用的构造说明（这种构造是不切实际的），而是通过对角化给出一个有积极意义的结论：不动点定理，以及它和病毒的联系。为了使讨论更为完整，我也会提供一个对角化的反面的较为典型应用，以显示即便给病毒一个非常广泛的定义，也不能保证一个病毒检测程序一定是安全的（无病毒散布）。

引言

由于两个原因，计算机病毒已经成了一个大众新闻的话题。第一，1988年一种广为人知的病毒（叫做“internet 蠕虫”）使工作在全国性的网络上的数量巨大的中型计算机受到感染。第二，很有一些病毒感染了个人计算机，这主要集中在分布于各个大学校园和计算机中心的 IBM 兼容机和 Macintoshes 计算机中。因此，许多使用 PC 的人都经历了性能下降的问题，这样就把很多时间浪费在清理病毒上，因而对广域网的连接表示怀疑。本文的目的是给出病毒的数学定义：一个可以“自我复制”的程序，并且指出了在由于任何计算领域的基本性质，病毒的存在是不可避免的。如果我们用更为操作性定义，就是说病毒是一个会传染和改变在其下去运行的操作系统，那样如果操作系统已经被某种病毒感染，任何一个病毒检测程序都不能担保一定安全。注意：这里明显的用到了对“病毒”的两个不同的定义。我将用来构造病毒的技术不是那种最直接可用的，但是我相信，它是很有趣的，也无需什么预备知识。这是 Kleene 的递归理论的一个例证性的使用，并且构造了对角化理论的一个正面形式。这里的这个想法实际上是 Kleene 的：“对角化不动点定理”我是在 Scott Weinstein 的课程中学到的——它是由 Owings^[2] 提出的。自复制机器和它们与递归理论的联系在 Rogers^[3] 做了一般性的广泛叙述。

记号

原题：Computer and Viruses: Diagonalization and Fixed Points 译自：Notices Of The American Mathematical Society, 37(1990), 858~861.

我们如何能够使病毒的定义：一个可以自我复制的程序更加形式化？让我们先来确定一下程序设计的领域，这就是一个程序的语言和支持它的平台（软件和硬件）。比如说语言是 Pascal，并且由你喜爱的编译器和硬件做平台。选择哪种语言 and 平台并不重要。为了避免不必要的细节，我们假设我们的 Pascal 程序从一个文件中读出一串字符，并把字符串打印到另一个文件中作为结果。现在，我们忽略 Pascal 程序和选择的处理器之间的交互作用，例如：试图除以零时在输出文件就会产生确定的字符串，等等。然而，一个程序是有可能无限循环的，在这种情况下，支持平台不会干预，并且这时程序不会返回值。（尽管在程序于循环中得到捕获之前或许已经写了一部分输出文件）表达式“ P 在输入 i 时挂起”，意思是程序 P 在处理输入 i 时无法在这样一个无限循环中得到捕获。这样的程序只有计算从字符文件到字符文件的部分功能。此外，还有一些程序有处理 N （自然数）的功能，也就是输入一串数字、返回一串数字。

让我们把 P 作为 Pascal 程序的集合，并且请注意，由于 Pascal 程序是从一个有限的字符表中取得给定的字符串，所以 P 是可数的。如果 $\{P_0, P_1, \dots\}$ 是 P 的一个索引，则用 ϕ_i 来表示由程序 P_i 所计算的（在字符文件上的）函数。 $P_i(y)$ 表示在输入 y 上运行 P 所得的结果，并且当 ϕ_i 存在时它也是 $\phi_i(y)$ 的一个表示，否则它就是没有定义的。正如上面所说，有的 ϕ_i 也会被看作 N 的函数，所以像 $\phi_{\phi_i(j)}$ 这样的表达式有时也会被用到。如果 $\phi_i(j)$ 没有确定一个自然数，那么将 $\phi_{\phi_i(j)}$ 看作全部的由那种在每个输入都会导致循环的 Pascal 程序所算出的未定义函数。

我们并不十分关心索引的精确的实质，我们关心的不过就是下面那些可被 Pascal 程序计算的函数：

1. $\text{Rep}: i \rightarrow P_i$;
2. $\text{Comp}: N \times N \rightarrow N$. 给出两个程序 P 和 Q 的索引， Comp 计算 P 和 Q 的复合程序的索引，复合程序是这样一种程序：首先程序通过输入给 Q ，然后一旦 Q 挂起的时候，在 Q 程序的输出结果上运行程序 P 。
3. 一个 Pascal 仿真程序，它有两个参数 P 和 i 作为输入。 P 是一个 Pascal 程序的（字符串）表示。Pascal 仿真程序将返回与程序运行在实际输入 i 下相同的结果（或者循环）。

所有的 Pascal 程序的字母列表都应该满足这些条件（尽管可以想象 Rep 以及 Comp 在运行时会消耗大量的时间。）

给出了一个索引和 Rep 函数后，我们可以形式的定义什么是一个可以自我复制的程序：

定义： 一个病毒就是一个这样的程序 $P = P_m$ ，它对于每个输入 y 来说，都有 $P(y) = \text{Rep}(m)$ 。

对角化和不动点的结果

对于大多数数学家所熟悉的对角化论证方法来说，他们都是得出否定式结论而为人所知。“不是 $[0,1]$ 内的所有实数都能表示成无穷二进位和 $\sum_{i=1}^{\infty} b_i 2^{-i}$ ($b_i \in \{0,1\}$)”和“不存在这样的集合 S 它的基数等于 S 的幂集的基数。”这两个结果，是由对角化方法所证明的。叫做 Cantor 定理。“没有一个 Pascal 程序可以决定集合 $\{i \in N | P_i \text{ 在输入}$

“时会挂起”的成员问题”又是另一个为对角化方法所证明的否定式结论。对角化理论的本质应该是这样的：如果 M_{ij} 是任意一个正方矩阵（甚至是无穷的），那么 M 的任意一行都不可能和对角线不相交，因为第 i 行会在位置 i 处与对角线相交。上面列举的种种被证明的否定式结论就是这样来证明的。其反面就意味着一个有一行在任何位置都与对角线不相交的正方矩阵的存在。“有些悖论的结果”也会以这种方式出现：假设有一个理发师 b 给任何一个不给自己理发的人理发，我们可以定义一个矩阵，如果 i 给 j 理发， $M_{ij}=1$ ，其他的为 0。并且在 $\forall j, M_{bj} = 1 - M_{jj}$ 的意义上，行 M_b 是对角线的非。那就是说，根本没有这样的理发师。

同样，也可以以肯定的方式用对角化方法获得一个不动点结果（参考 Owings [2]。）假设 f 是一个函数，如果一个正方矩阵 M 的每一行都与一些行有 f -相关，那就是说，如果 $\forall i \exists k$ ，使得 $\forall j, f(M_{ij}) = M_{kj}$ ，成立，我们就称 M 在 f 下为行方式封闭（row-wise closed）。此外，如果 M 的某一行和对角线相等，那么就称 M 是对角线完备的。令人惊讶的是，有这样两个属性的 M 的存在，就保证了 f 有一个不动点。为了了解这一点，假设 M 与角线相等，行 k 是行 i 的 f -映像， $f(M_{ik}) = M_{kk} = M_{ik}$ 。那就是说， M_{ik} 在 f 下是不动的。这个结果是“对角不动点引理”。

现在，假设 $f: N \rightarrow N$ 是一个函数，由某个 Pascal 程序计算，这与我们的 Pascal 程序的索引有关： $\phi_i = \phi_j \Rightarrow \phi_{f(i)} = \phi_{f(j)}$ 。在这个情形中 f 叫做“扩展函数”。由于 f 是扩展函数，它在函数 ϕ 上定义了一个算子： $f(\phi_i) = \phi_{f(i)}$ （也用 f 表示）。我们将会通过使用对角化不动点引理证明，存在一个 $n \in N$ 使 $\phi_n = \phi_{f(n)}$ 。让 M 是由部分函数作成的无穷矩阵： $M_{ij} = \phi_{\phi_i(j)}$ 。注意，每次输入的 M_{ij} 都可以被一些 Pascal 程序 P_i 所计算。实际上，这样的 i 可以由 i 和 j 来有效的确定。 P_i 是这样一个程序：1) 使用 Rep 构造 P_i ；2) 用一个 Pascal 仿真子程序来确定 P_i 对输入 j 的作用；3) 一旦第二步完成，使用 Rep 再构造 $P_{\phi_i(j)}$ ；4) 就像第三步，再使用 Pascal 仿真程序计算第三步所构造的程序对它的 (P_i 的) 输入的作用。

我们需要证明 M 是对角化完备的和在 f 下封闭的。假设 P 是一个关于任意输入 $j \in N$ 的一个 Pascal 程序，进行与 $P_j(j)$ 相同的计算。（就像上面一样，也需要调用 Rep 和 Pascal 仿真器）。假设 k 是 P 的索引，那么对于所有的 $j \in N$ $\phi_k(j) = \phi_j(j)$ ， $\phi_{\phi_i(j)} = \phi_{\phi_j(j)}$ 。所以 M 的 k 行与对角线相等。现在我们证明 M 是在 f 下封闭的。假设 P 是一个（依赖于输入 i ）Pascal 程序取输入 j 并进行以下操作：1) 计算 Rep(i)；2) 使用 Pascal 仿真程序使第一步的结果作为输入 j ；3) 一旦第二步完成，计算 f 在第二步的结果上的结果。令 $k = k(i)$ 为 P 的索引。因此除非 $\phi_i(j)$ 有循环，否则 $\phi_k(j) = f(\phi_i(j))$ ，这种情况下所有的表达式都是不确定的。那么对任何 j 来说：

$$f(M_{ij}) = \phi_{f(\phi_i(j))} = \phi_{(\phi_k(j))} = M_{kj}$$

那就是说，在 f 下 M 是行方式封闭（row-wise closed）的。因为在计算 k 中 f 是一个可计算的 Pascal 程序，并且还需要有 f 是扩展的，这样表达式 $f(M_{ij})$ 才是有意义的。在 f 满足这些条件和先前对索引所做的假设条件下，我们可以用对角化不动点引理推断一个递归定理的弱形式：存在一个 $n \in N$ 使得 $\phi_n = \phi_{f(n)}$ 成立。我们说，仅仅使用 Comp, Rep 和 Pascal 程序仿真器，我们可以从 f 完全构造出一个 n 。

可以这样强化递归定理使得 f 为扩展的这个限制去掉. 令 f 为 N 上任意 Pascal 可计算的函数. 我们说 ϕ_i 和 ϕ_j 是 $\bar{f}$ -相关, 如果 $\exists k$ 使得 $\phi_i = \phi_k$ 以及 $\phi_j = \phi_{f(k)}$. 那么我们上面所做的论证就可以证明, M 的每一个行 i 是与 M 的某一行 k $\bar{f}$ -相关—— k 的构造和上面一样. 由于对角线是 M 的一个行, 某一个对角线元素 M_{ii} 就和它自己 $\bar{f}$ -相关的. 因此, 存在一个 k 使得 $\phi_k = M_{ii} = \phi_{f(k)}$.

我们现在来说明像上面定义的病毒可以怎样来构造. 令 r 是 Rep 的索引, 并令 $f: N \rightarrow N$ 为函数, 该函数对给定的一个索引 x , 返回一个 Pascal 程序 P 的索引, 其工作如下: 1) 计算 $\text{Comp}(r, x)$; 2) 返回这个值, 与它 (P) 的输入无关. (所以 f 总是返回计算常数函数的程序的索引, 但这个常数函数依赖于 x 的值.) 很清楚, f 通过一些 Pascal 程序计算, 假设 Comp 是这样的. (可见无论如何 f 都不是扩展的.) 根据递归理论, 存在 n 使得 $\phi_n = \phi_{f(n)}$, 它是返回 $\text{Comp}(r, n)$ 的 y 的函数. 因此:

$$\forall y \text{Rep}(\phi_n(y)) = \text{Rep}(\text{Comp}(r, n))$$

$$\forall y P_{\text{Comp}(r, n)}(y) = \text{Rep}(\text{Comp}(r, n))$$

病毒是一个带有 $\text{Comp}(r, n)$ 索引的程序: 在输入任意 y 时, 它的输出就是代表它自己的字符串.

我所描述的整个过程是行之有效的: 原则上, 我们可以以这样的方式编写一个计算机病毒. 当然, 实际上病毒并不是这样写出来的, 但它的确证明了自我复制程序中的普遍存在性. 证明 Pascal 病毒的存在只需做很微弱的假设, 其原因在于: 对任何语言, 我们都可以为它编写一个解释程序并且易于操作程序表示 (Comp 和 Rep). 在构造的核心部分有一个替代函数 (f), 它的不动点被用作建立病毒. 实际中的病毒 (见 [4]) 使用替代函数 f 和不动点和我们这里使用 f 和不动点的方式完全一样——但是它的构造可能相当简单.

病毒的检测: 一个对角化的否定结果¹⁾

让我们回到稍微现实一点儿的环境.² 程序运行于现代计算机上, 不像那些我们到此为止所描述的那样运行于“一个环境”中. 这就是说, 当程序执行时, 是作为一个逻辑独立程序的子程序来运行的, 这个独立程序就是操作系统, 是负责进行一些管理, 例如一次和二次存储管理, 进程管理, 记录统计等等, 为达到实际目的, 上述病毒通过“感染操作系统”达到它们的破坏作用. 那就说, 它们在把自己复制到操作系统内.

为了展现其恶毒, 在剩余的篇章中, 我们将使用更为现代的“病毒”的定义:

定义: 一个病毒是这样一个程序, 一旦运行, 就改变操作系统的代码.

(当一个操作系统的新版本合法出版, 它就是替换, 而不是改变先前的操作系统.) 改变操作系统的特定作用我们不必知道: 自我复制的要求对于我们所求的结果也不是必要的.

如果可以通过把程序提交给一个过滤程序, 就知道哪些程序是病毒, 哪些不是病毒, 是一件好事, 这样就可以避免那些无意的和可能给操作系统带来伤害的改变所带来

¹⁾ 下面的材料来自于 Dowling[1].

的损失和麻烦。我们现在来说明，没有程序可以完全胜任对所有的可能的输入，能担保不会扩散病毒。

我们从固定一个操作系统 OS 开始，并做一个定义：

定义： 程序 P 透过输入 x 传播病毒，如果在操作系统 OS 上运行 P 并输入 x 将改变 OS 否则，对输入 x 操作系统就是安全的。如果对于所有的输入都是安全的，那么就说程序是安全的。

我们还做一个假设；那就是 OS 中已经存在病毒，否则我们就没有测试的必要了。现在，为了得出矛盾的结果，让我们假设有一个安全程序 IS-SAFE 负责判断在任意输入下的任意程序 P 的安全性。因此：

$$\text{IS-SAFE}(P, x) = \begin{cases} \text{Yes} & \text{if } P \text{ is safe on input } x \\ \text{No} & \text{otherwise} \end{cases}$$

有了这样一个程序，并且我们假设存在病毒，很容易写下一个只有一个变量的程序 $D()$ 有以下功能：

$$D(P) = \begin{cases} \text{Write "Have a nice day"} & \text{if IS-SAFE}(P, P) = \text{no} \\ \text{Alter OS} & \text{otherwise} \end{cases}$$

不严格的讲， D 所有的元素与矩阵 $M_{P,x} = \text{IS-SAFE}(P, x)$ 的对角线的各位置元素都是不同的。

我们现在来证明，通过检查 D 在输入 D 下的功能，IS-SAFE 不可能同时安全又是正确的。如果 D 在输入 D 时是安全的，这可能仅仅因为它没有执行其他的子句，那就是，因为 $\text{IS-SAFE}(D, D) = \text{"no"}$ ，这就说明了 IS-SAFE 是不正确的。另一方面，如果在 D 在输入 D 时改变了 OS，那有两种可能性。一种可能，在调用 $\text{IS-SAFE}(D, D)$ 时或许返回了 "yes"，所以 D 中的其它子句被执行了，这种情况下，IS-SAFE 是不正确的。另一种可能，如果 IS-SAFE 返回了 "no" (D 就简单的打印 "Have a nice day")，那么假设 D 在输入 D 时是不安全的就意味着对 IS-SAFE 的调用一定是其故障所在，也就是说 IS-SAFE 不是安全的。因此我们得出这个结论，存在一个安全而又正确的程序在 OS 上运行，并且，在检查输入时是安全的假设一定是不正确的；这样就没有这样一个程序 IS-SAFE 存在了。

我们假设 OS 存在病毒，这在证明 IS-SAFE 不存在是必要的，但绝不一定适用于任何一个操作系统。一个操作系统，甚至是一些它所运行的程序，可能是存储在计算机的不可写内存区域上的。在这种情况下，没有任何程序的执行会导致操作系统的改变，那么有一个安全程序 IS-SAFE 计算常量函数 "yes"。例如，一些个人计算机操作系统固化在 ROM(只读内存)中，和在一些内嵌系统(如控制录像机的计算机和汽车的点火系统)中的所有 OS 和运行的程序都在 ROM 中。这种情况的可能的模式是一个程序 OS 映射三元组到三元组： $\langle P, x, OS \rangle \mapsto \langle P, P(x), OS \rangle$ 。OS 不仅仅计算 $P(x)$ ，还要返回 OS 和 $P(\text{unchanged})$ 。类似上面所做的分析，导致了这样的程序的存在，它计算这样的函数：按照定义它不会被感染(因为它返回 OS，而不是某种改变了的 OS)。

然而，对于通用的计算机来说，OS 和 P 都放在计算机的存储设备，主内存(虚拟的或者是物理的)的同一块物理区域，并且这块区域是可写的。(下转 255 页)

Saharon Shelah

Saharon Shelah 许多年以来一直是数学基础和数理逻辑的领头人物。他的惊人的 700 篇论文和半打的专著包含了几个全新的理论, 彻底改变了模型论和现代集合论的进程, 并且为解决许多其它数学分支的老问题提供了方法, 包括群论、拓扑学、测度论、Banach 空间以及组合学。Shelah 建立了集合论的许多分支, 如著名的真强迫理论和可能的共尾度理论 (theory of proper forcing and the theory of possible cofinalities)。这是对基数概念的惊人的精细, 它导致了以前认为远远超出不可判定性范围的不定陈述的证明。他在集合理论代数及其应用方面的工作表明了代数的许多部分具有不能被集合论中的万有识别公理控制的现象。在模型论中, 他完成了被称作“稳定性理论”的深刻的结构分析的不朽纲领, 现在它已主宰了该领域的绝大部分。

Shelah 1945 年出身于以色列的 Jerusalem。他于 1964 年在 Tel Aviv 大学获学士学位, 1967 年和 1969 年在 Jerusalem 的 Hebrew 大学分别获得硕士学位 (M.Sc.) 和博士学位 (Ph.D.)。他 1969-70 在 Princeton 大学工作, 1970-71 在 Los Angeles 工作, 之后回到 Jerusalem 的 Hebrew 大学任教授, 直到现在。自 1986 年起他一直是 Rutgers 大学的兼职教授。他以前获得的荣誉包括 Erdos 奖 (1977)、Rothschild 奖 (1982)、符号逻辑协会的 C. Karp 奖 (1983)、SIAM 的 George Pólya 奖 (1992)、Israel 的数学研究奖 (1998)、日本数学会奖 (1999), 以及匈牙利科学院的 János Bolyai 奖 (2000)。他是以色列人文和科学学院的院士、美国艺术和科学学院的院士。

(邹建成 译 李培信 校)

(上接 253 页)

即使是在 OS 的控制之下的 P 的执行, 也可能导致 OS 存储的内存位置的变化。此外, 出于实际应用的原因, 一般操作系统在程序执行时都不会写它们自己, 而是计算上述三元组上的第二部分, 把这些因素综合起来考虑, 就得出了可能存在病毒的问题。并且证明了我们的假设, 该假设导致 IS-SAFE 不存在。

参考文献

- 1 Dowling, William F., "There are no safe virus tests," *Am. Math. Monthly*, 1996.9, pp 835-6
- 2 Owings J. C., "Diagonalization and the recursion theorem," *Notre Dame Journal of Formal Logic*, 1973, 14: Number 1, pp. 95-99.
- 3 Rogers H., *Theory of Recursive Functions and Effective Computability*, McGraw-Hill, 1967.
- 4 I. Witten, Computer (in)security: infiltrating open systems, *Abacus* 4. (Summer 1987) 7-25

(李长昕 译 培廉 校)